FREE PROMPT PACK · CLAUDE CODE · MULTILINGUAL

Your Jarvis, in *Sixteen* Languages

Twelve copy-paste prompts in two parts. Part One builds the Jarvis from my videos: a galaxy of your own notes, a brain that answers from them, a voice both ways, a butler, eyes on your screen, and a preflight that keeps it honest. Part Two makes him multilingual: switch him to Spanish, German or Japanese in any phrasing, keep talking English while he answers in the language you chose, hear every line he says in that language down to the smallest "one moment, sir", and point him at a brochure you cannot read.

ZUBAIR TRABZADA

AI WORKSHOP

OCTOBER 2026

How to use this pack

What you need: Claude Code (I build with Claude Opus 5.5; type /model to pick it), Google Chrome for the microphone and speech, a folder of markdown notes, and an Anthropic API key for the brain. Part Two needs nothing new: the translation layer runs on the same key with the cheapest Claude model. Optional: an ElevenLabs key if you want one voice that speaks every language.

Set up in one line. Open a terminal, make an empty folder, and start Claude Code inside it: `mkdir my-jarvis && cd my-jarvis && claude`. Then paste Prompt 01.

1 **Part Two assumes Part One.**
The language prompts bolt onto the server and viewer that Part One creates. If you already built Jarvis from an earlier pack, skip straight to Prompt 07; the prompts refer to "the server" and "the viewer" and Claude Code will find them.

2 **Paste each prompt whole, unedited, the first time.**
Every prompt is a complete brief: what to build, the laws it must respect, and how to prove it works. The verification at the end of each one is where the reliability comes from. Run it as written before you customize.

3 **Never paste an API key into a chat window.**
Prompt 01 creates a placeholder in a config file. You type the real key into that file yourself, in your own editor. Treat anything pasted into any chat as exposed.

4 **English mode never changes.**
Every prompt in Part Two does nothing at all while the language is English: no extra calls, no extra waiting, no new behaviour. If English ever gets slower after a Part Two prompt, that is a bug. Say so and have it fixed.

5 **Test in a muted tab, then put the language back.**
Every test tab opens with ?mute=1 so you never hear him from the background, and every test that switches the language restores it at the end. A test that leaves your Jarvis speaking Polish is a failed test.

What multilingual mode is, honestly

A translation app turns one language into another. This is something else: an assistant that lives in your language. He answers from your notes, reads your screen, jokes and confirms in the language you picked, while you keep talking in whichever language you like. Here is what that is and what it is not.

	A TRANSLATION APP	YOUR MULTILINGUAL JARVIS
What it does	Turns a sentence in one language into another.	**Lives in your language.** Answers, quips, confirmations and errors, all in the language you chose.
What you speak	Each person speaks their own language.	**Whatever you like.** Speak English and hear Spanish back, or switch his ear too with one sentence.
What it knows	Only what you say to it.	**Your notes and your screen.** Ask in English about a Spanish brochure and hear the answer in German.
Languages	Often seventy or more.	**Sixteen in this build**, each with a native voice when your computer has one. Adding another is one line.
What it costs	Free, or a subscription.	**Your normal brain**, plus a fraction of a cent the first time a fixed line is translated. Repeats come from a cache and cost nothing.
What it cannot do	Know your business.	**Live interpreting.** He talks with you; he does not sit between two people translating a conversation both ways. And his ear is only as good as the browser's recognizer for that language.

What is not the model. Claude writes every answer in your language because the system prompt tells it to, and it is very good at it. The switch, the separate ear, the per-language voice, the translation cache and the chip are mechanics you build in Part Two. The model supplies the words. The design is yours.

Part One · the foundation

Six prompts that build the Jarvis from the videos: a 3D galaxy of your own notes, a brain that answers only from them, a voice in both directions, the camera dive to the source note, a personality, eyes on your screen, and a preflight harness that tells you the truth about what works.

If you built Jarvis from an earlier pack, you already have all of this: skip to Part Two. Prompt 05, the eyes, is the one Part Two leans on for the brochure demo in Prompt 11, so add it now if your build does not have it yet.

PROMPT 01 · THE FOUNDATION

The Galaxy and the Brain

Best for: the first paste. One prompt builds the whole foundation and starts it.

```
Build me a 3D knowledge galaxy from my markdown notes, and give it a brain I can talk to. Do the whole thing in one pass, then start it and verify it before you tell me it's done.

MY NOTES: [PASTE THE FULL PATH TO YOUR NOTES FOLDER, or delete this line and write "I have no notes, so generate 30 realistic sample notes about a small business into ./notes first"]

THE INDEXER: build.py, Python 3, standard library only. Scan every .md file and write viewer/graph-data.js containing const GRAPH = {nodes: [...], links: [...]}. Each node: a label from the filename, its parent folder as a group, roughly a 700-character excerpt. Link two notes when one mentions the other's title or they share [[wikilinks]]. CRITICAL: every node gets a numeric id equal to its index in the nodes array. Later prompts look nodes up by index.

THE VIEWER: viewer/index.html, single page, 3d-force-graph from a CDN pinned to an exact version. No npm, no build step, no framework. Cinematic: black space, a starfield, nodes glowing and colour-coded by group, slow idle rotation. Clicking a node flies the camera to it, lights its neighbours, and opens a side panel with the excerpt.

THE SERVER: server.py, port 4700, bound to 127.0.0.1, serving ONLY the viewer/ folder. Create config.json in the PROJECT ROOT, never inside viewer/, containing {"anthropic_api_key": "PUT-YOUR-KEY-HERE", "model": "claude-opus-5-5"}. Do not ask me for the key and never put it anywhere the browser can reach.

THE BRAIN: add POST /chat. Score every note against my question by keyword overlap, weighting title matches higher, take the top 6, and call the Anthropic Messages API with the official anthropic Python package and the model from config.json. System prompt: answer ONLY from these notes, in two or three sentences, and say so plainly when the notes do not cover it. Return {"answer": "...", "nodes": [array indexes used]}. Keep a short conversation history server-side so follow-ups work. Add a clean input bar at the bottom of the viewer that shows the answer.

OPUS 5.5 RULES: thinking is always on for this model, so never send a thinking-disabled setting, and never force a tool choice; both are rejected. Effort defaults to medium; set it per route in output_config. max_tokens covers the thinking AND the answer together, so give chat at least 2000 and retry once with a bigger budget if the text comes back empty.

FINISH PROPERLY: start the server, confirm the page loads and the graph renders, confirm /chat returns a clean placeholder-key error rather than a crash. Then tell me exactly what to paste into config.json and give me three example questions based on my actual note titles.
```

Make it yours: casual chat is happy at low effort and noticeably snappier. Save higher effort for the heavy jobs, where getting it right matters more than a second of speed.

PROMPT 02 · SPEECH

The Voice, both directions

Best for: the moment it stops being a website and starts being an assistant.

```
Give it a voice, both directions, using only what the browser already has. No paid speech services.

SPEAKING: every answer is spoken with the Web Speech API. Prefer a British English voice when the system has one. Browsers block audio until the user has interacted with the page, so make the first spoken line fire reliably after my first click, never before.

LISTENING: a microphone button using webkitSpeechRecognition. I click it, I speak, the transcript goes through the same /chat flow as typing.

THE PART EVERYONE GETS WRONG: speech recognition finalizes a phrase when I pause, and I pause in the middle of sentences. Do NOT dispatch on the first final result. Buffer it, with the wait in ONE named constant at the top of the file, FINISH_MS, set to 900. If more speech arrives before the timer fires, append it and restart the timer; the pause that truly ends my thought sends the whole combined sentence. Short interrupt words like "stop" and "wait" must bypass the buffer and fire instantly.

ONE PLACE EACH: all speech goes through ONE speak function, the voice is chosen by ONE function, and every recognizer reads its language from ONE variable. A later prompt will make all three follow a language setting, and it must not have to hunt for them.

THE MUTE FLAG: if the page URL contains ?mute=1, nothing in that tab ever speaks. I will use that for testing so I never hear the assistant from a background tab. Put the check inside the one speak function, so nothing can route around it.

STATUS: a small status line that shows listening, thinking and speaking, so I always know which of us the machine thinks is talking.

VERIFY: give me a test plan: what to click, what to say, and what I should see and hear, including one deliberate mid-sentence pause to prove the finish window works, and one ?mute=1 tab to prove the silence.
```

Make it yours: FINISH_MS is the single most personal number in this build. Too low and it cuts you off mid-thought; too high and it feels sluggish. Start at 900 and tune it after a day of real use, never in one sitting.

PROMPT 03 · THE DEMO MOMENT

Make it prove where the answer came from

Best for: the ten seconds that make people lean toward the screen.

```
When it answers from a note, I want the galaxy to prove it.

/chat already returns a nodes array of the indexes it used. Use it:

- Fly the camera to the top source node, light it and its direct neighbours, and open its side panel while the answer is still being spoken, so I can see exactly which note it came from.
- If the answer drew on four or more notes, do not fly anywhere. Light the whole cluster. Flying to one arbitrary node out of six is a lie about where the answer came from.
- Never read the note aloud unless I ask. It is on screen. The spoken answer stays short.
- Small talk must not drag the camera around. "Good morning" and jokes hold the graph still. Only real questions about my notes move the camera.

That last rule separates a demo from a toy, so implement it deliberately: decide whether the question was about my notes before you decide whether to move the camera.

ON REQUEST: when I say "pull up my notes on [topic]", open them with one short line, something like "I've pulled up your notes, sir. Say read it out loud and I shall." When I say "read it out loud", read the top note's excerpt.

Then show me: one question that flies to a single note, one that lights a cluster, one that must not move the camera at all, and one "pull up" followed by "read it out loud".
```

Make it yours: this is the shot worth recording. Ask something whose answer lives in one very specific note and film the dive. It reads on video better than any feature list.

PROMPT 04 · CHARACTER

The Butler

Best for: turning a question box into something people want to talk to.

```
Give it a personality. Rewrite the /chat system prompt so the assistant is a dry, impeccably polite British butler with a razor wit. Rules:

- Address me as "sir", occasionally, not in every sentence. Over-using it is the difference between charming and grating.
- Answer in ONE witty sentence plus the facts. Never recite the note back; it is on my screen.
- One genuinely funny line beats three bland ones. If nothing funny is available, be brief instead of forcing it.
- When my notes do not cover something, say so plainly and with a little dignity. Never invent a source, never pad, never pretend a related note is the answer.
- Handle small talk without dragging the camera around the graph.

BOOT GREETING: when the page loads or refreshes it says one short line, "Hello, sir." and nothing more. Show the real indexed note count ON SCREEN, read from the graph data, never hardcoded.

THE FIXED LINES: every line he says WITHOUT asking the brain (the greeting, "one moment, sir", "I've pulled up your notes", every error and acknowledgment) lives in ONE clearly labelled table in the viewer, never scattered as strings through the code. A later prompt will translate that table, and anything outside it will stay in English forever.

Put the entire persona in ONE clearly commented block at the top of server.py so I can rewrite the character later without hunting through the file.

Then give me five things to say to it that show off the personality, including one question it definitely cannot answer from my notes.
```

Make it yours: the butler is just the default. Swap the persona block for a terse mission-control operator, a sarcastic friend, or a patient teacher. Keeping it in one block is what makes that a thirty-second change, in every language.

PROMPT 05 · SIGHT

The Eyes on your screen

Best for: the first time it answers about something you never described to it.

```
Give it sight. I want to point at my own screen and ask "what do you think of this?"

- A screen button that starts a getDisplayMedia share and holds the stream. While a share is live, show a small,
unmistakable indicator. This is not a feature to be subtle about.
- When I ask a question while sharing, grab ONE frame from the stream to a canvas at the moment I ask, encode it as
JPEG, and send it to a new POST /see endpoint with my question.
- /see calls the same Claude brain from config.json with the image and the question. Tell it to answer specifically
about what is on screen, headline first, and to say plainly when the frame is too small or blurry to read, rather than
guessing.
- The answer comes back through the same voice and on-screen path as every other answer.

BUILD IT SO IT CANNOT LIE TO ME:
- The frame is captured when I ask, never cached from when the share started.
- If the share has ended, it says so. It never answers a screen question from memory of an earlier frame.
- Encode JPEG and send image/jpeg. Match the media type to what you actually encode, or the endpoint fails in a way
that looks like the whole feature is dead when it is one wrong string.
- Capture at the screen's full resolution. Small print on a brochure or a web page is unreadable from a shrunken frame,
and a later prompt reads exactly that.

Verify end to end in front of me: start a share, ask "what am I looking at", and show me the real answer.
```

Make it yours: screen sharing is far more useful day to day than a webcam and much easier to trust. Point it at a spreadsheet, an error message, or a page in a language you cannot read, and ask what it says.

PROMPT 06 · THE ONE THAT MATTERS

The Preflight

Best for: the difference between a weekend demo and something you rely on.

```
Build me a preflight harness. This is the most important prompt in this pack, so take your time.

Write preflight.py that runs every LIVE chain end to end and prints a verdict. Not unit tests, not mocks. Real calls
against the running system, because the failures that hurt are the ones where every unit test passes and a real chain
is dead.

Check, in order, with a tick or a cross for each:
- The server is up and serving the viewer.
- The graph data loads and the node count is greater than zero.
- /chat returns a well-formed answer to a real question, with a nodes array.
- The API key in config.json is valid, by one real minimal call.
- The model named in config.json is one the key can actually reach: ask the Models API for it by id, and fail with the
id in the message.
- /see answers a real JPEG. Send the media type the client actually sends; a PNG probe can fail and mimic a dead
endpoint.
- The files the browser is actually SERVED match the files on disk. A stale served file is the most common cause of "I
fixed it but nothing changed".
- config.json is not reachable from the browser. This one must fail loudly.
- A ?mute=1 tab really is silent: the one speak function refuses when the flag is set.

Finish with one summary line: N pass, N fail, N warn. Exit non-zero on any failure.

Then run it and show me the output. If anything fails, fix the cause and run it again until it is clean. Do not
describe what it would say. Run it.

From now on, whenever you tell me a change is done, run preflight.py first and paste the summary line. "Done" means
preflight passed. Every prompt in Part Two will add checks to this file.
```

Make it yours: add a check every time something breaks on you. A harness that grows one check per real incident becomes the most valuable file in the project within a month. Mine has 57 and every one was earned.

Part Two · the languages

Six prompts that make your Jarvis multilingual. The first is the switch: sixteen languages, any phrasing. Then the voice and the chip, the translation layer that catches every fixed line, the ear that lets you keep speaking English, the brochure demo, and the drill that proves all of it before you film.

The switch	Any phrasing, in any language, by a voice route or by the brain itself. An unknown language gets an honest refusal, never the nearest guess.
The tongue	The language he answers in. One non-negotiable line in the system prompt, on every reply.
The ear	The language he listens in. English by default and separate from the tongue, so you can speak English and hear Spanish.
Every line	The fixed lines he says without the brain, translated once by the cheapest model and cached forever.
The voice	A native voice per language when your computer has one, and a small chip that always shows which language is live.

Two laws hold across all six prompts. English mode never changes: no extra calls, no extra waiting. And nothing is translated twice: anything the brain wrote is already in the right language, so it goes straight to your ears and your screen.

PROMPT 07 · THE SWITCH

Sixteen Languages, One Switch

Best for: the first paste of Part Two. The table, the state, the tongue, and every way to flip it.

```
Make my Jarvis multilingual: sixteen languages, switchable by voice, with English as the default that never changes.

THE TABLE: ONE dictionary near the top of server.py, LANGS: code to (English name, BCP-47 locale, native name, a one-line in-character confirmation written in that language). English, Spanish, French, German, Italian, Portuguese, Dutch, Hindi, Urdu, Arabic, Japanese, Korean, Chinese, Russian, Turkish, Polish. Beside it an alias map: every English name, the native names (español, français, deutsch, italiano, português, with and without accents), "mandarin" for Chinese, and "default" and "usual" for English. A new language must be one line in each.

THE STATE: lang.json in the project root holds the active code. GET /api/lang returns the code, the name, whether it is the default, and the full list. POST /lang {lang} accepts any alias and returns the confirmation line. An unknown language returns 404 with an in-character refusal that lists the repertoire. NEVER fall back to the nearest match.

THE TONGUE: when the language is not English, append to the system prompt: "LANGUAGE (non-negotiable): EVERY reply, answers, quips and acknowledgments, is written entirely in {name}, in the same character." In English mode append instead: reply in English, but if I write to you in another language, reply in that language.

THE TAG: tell the brain that when I ask it to speak or switch to a language, in ANY phrasing and in ANY language ("habla español", "parle-moi en français", "back to English"), it replies with exactly [LANG:language name in English] and nothing else. The server executes the tag and the viewer speaks the confirmation. Phrasings nobody planned for still work.

THE FAST ROUTES: in the viewer, before anything reaches the brain: "speak Spanish", "respond in French", "switch to German", "back to English", "what languages do you speak". "Switch to" accepts ONLY real language names, so "switch to Opus" stays a brain swap and anything else that starts with "switch to" keeps working. The viewer re-reads /api/lang every 20 seconds and after every answer, so a switch made by the tag catches up.

VERIFY: switch to Spanish, ask a question in English and paste the Spanish answer. Ask for Klingon and paste the refusal. Show that every other "switch to" command still does its old job. Go back to English. Add all of it to preflight.
```

Make it yours: the confirmation lines are the first thing a native speaker hears. Have Claude write them in character, then run each one past someone who speaks the language before you film for an audience that does.

PROMPT 08 · THE VOICE

A Voice for Every Language, and the Chip

Best for: making him sound right in Japanese, and always showing which language is live.

```
Make him sound right in every language, and always show which one is live.

THE VOICE: the ONE function that picks the speaking voice now reads the active language. Pick the best installed voice
for its locale: the exact locale first (es-ES), then the language alone (es), and among those prefer voices named
Google, Premium or Enhanced. If this computer has no voice for that language, fall back to the default voice and say so
ONCE on screen ("No Japanese voice installed on this computer"), never silently. If config.json has an ElevenLabs key,
use a multilingual model so the SAME voice speaks every language, and keep the browser path as the fallback.

THE CHIP: a small pill that shows the native name of the active language (Español, Deutsch, 日本語), visible only when
the language is not English. Give it its OWN fixed slot on screen. Never put it inside another control's layout.
Measure it against every element around it and prove nothing overlaps: mine was hidden under a row of buttons the first
time.

INSTANT BOOT: cache the active language in localStorage and apply it before the first /api/lang round trip, so the
first sentence after a reload already uses the right voice. The server stays the source of truth; when they disagree,
the server wins.

THE MUTE LAW STILL HOLDS: a ?mute=1 tab never speaks, in any language. The check stays inside the one speak function.

VERIFY: print a table of all sixteen languages with the voice this computer would use for each, and mark the ones that
fall back. Then, in a ?mute=1 tab at 1440 by 900, switch to German, and paste the chip's bounding box beside the boxes
of the controls around it. Switch back to English.
```

Make it yours: on a Mac, better voices are free. System Settings, Accessibility, Spoken Content, then Manage Voices next to System Voice: download the Premium or Enhanced voice for each language you plan to demo, then rerun the table.

PROMPT 09 · EVERY LINE

Every Line in Your Language

Best for: the moment a native speaker stops noticing it was built in English.

```
The brain now answers in my language. Everything else he says still does not: "one moment, sir", "I've pulled up your notes", the errors, the note he reads back. Fix all of it, without ever translating the brain twice.

THE TRANSLATOR: POST /xlate {text, to} on the server; "to" defaults to the active language. If the target is English and the text is already English, return it untouched and make no call. Otherwise ONE call to claude-haiku-4-5, the fastest and cheapest Claude, with this system prompt: translate into {name}; it is a line spoken by a dry British butler to the man he calls sir, so keep the tone and render "sir" with the natural polite form; keep names, product names, numbers, prices, dates, URLs and email addresses unchanged; anything inside quotation marks is a command I say out loud, so keep it in English exactly; output only the translation. Cache every result on disk keyed by language plus text, capped at 3000 entries. Any failure returns the original line. Never silence.

NATIVE MARK: everything that comes back from the server (the brain's answers, the confirmation lines) is already in the right language. Mark it native the moment it arrives and never send it to /xlate.

THE VIEWER: the function that shows a line uses the cached translation instantly when there is one; otherwise it shows the English line and swaps in the translation when it arrives, but only if that line is still on screen. The ONE speak function translates first, then speaks. When the language changes, and five seconds after boot, pre-translate the whole fixed-lines table from Prompt 04 so the common lines are instant.

VERIFY: in a ?mute=1 tab with the language set to German, pull up a note and paste what appears and how many milliseconds it took, first time and cached. Then ask one real question and read the network log: the brain's answer must cause zero /xlate calls. Put the language back to English.
```

Make it yours: the quotation-mark rule matters more than it looks. "Say 'read it out loud' and I shall" must keep the English command, because his ear is still listening in English (Prompt 10). Translate the command and you have told yourself to say something he will not understand.

PROMPT 10 · THE EAR

Speak English, Hear Spanish

Best for: talking in your own language while he answers in someone else's.

```
Split what he listens in from what he answers in. I want to say: "Jarvis, I'm going to speak in English and you respond to me only in Spanish." Then I keep talking English, and every reply comes back in Spanish.

THE EAR: every speech recognizer in the viewer reads its locale from ONE variable, the ear, never from the reply language. The ear is English by default and stays English when the reply language changes. Store whether the ear follows the reply language as one flag in localStorage.

THE ROUTES, checked before the brain:
- Reply only: a sentence aimed at him, "you respond / reply / answer / talk / speak (back) (to me) (only) in X", matched ANYWHERE in the sentence, switches the reply language and leaves the ear in English.
- Ear on: "listen in Spanish" or "I'll speak to you in French" turns the flag on and switches the reply language too.
- Ear off: "listen in English" turns the flag off. The ear comes back to English and the reply language stays where it is. Confirm in one line, translated.
- Not a switch: "How should I respond in Spanish to this client email?" is a question for the brain. Only sentences aimed at HIM ("you respond...") may switch anything.

WHY: a recognizer set to Spanish while I speak English mangles every sentence. The reply language is a preference. The ear has to match the mouth.

VERIFY: a table of eight sentences with the expected route and the actual one: both phrasings above, "listen in German", "listen in English", "speak Spanish", the client-email question, "switch to Opus", and "what languages do you speak". Then live, in a ?mute=1 tab: my sentence must call /lang and NOT /chat, and the ear must still read English afterwards. Paste both, then put everything back to English.
```

Make it yours: if you are bilingual, flip it: "listen in Spanish" and answer in English, so you practise speaking while your notes come back in the language you read fastest.

PROMPT 11 · THE DEMO

Read Me Anything on Screen

Best for: the shot. A brochure you cannot read, read for you.

```
Now the demo. I share my screen with a page in a language I cannot read, say a tourism brochure for Valencia in
Spanish, and Jarvis reads it for me. Build on the eyes from Prompt 05.

THREE ASKS MUST WORK:
- "What does this say?" A short summary of the page in the CURRENT reply language, headline first.
- "Read me the headline in the original." Just that one line, in the page's own language, spoken with a voice for THAT
language. /see returns {answer, lang}, where the brain names the language its answer is written in, and the speak
function picks the voice from lang, not from the reply setting. An English voice reading Spanish ruins the shot.
- "What's the one thing I can't miss?" Answered from the page itself, not from general knowledge, naming the part of
the page it came from. If the frame is too small to read, say so instead of guessing.

THE LOCK: once I switch the reply language (Prompt 07 or 10), it holds. Locked to German, "translate this into English"
returns the English sentence itself, with everything around it in German. "Say that in English" gets a polite refusal
in German that tells me the exact words to say: "back to English". Write that rule into the system prompt so he never
half-switches, and never points me to a settings screen that does not exist.

CAPTIONS: "captions on" puts a small, muted English line under every non-English answer, using /xlate with to set to
English, so a viewer who speaks neither language can follow. "Captions off" removes them. Off by default.

VERIFY: open any public tourism page in Spanish, share it, and run the three asks. Then say "I'll speak English, you
answer only in German", ask in English "what's the best time of year to go?", and paste the German answer with its
caption. Put everything back to English.
```

Make it yours: film it in this order: read and translate first, lock second. Once he is locked to German, the translation beat comes back mostly in German, which is correct, and confusing on camera.

PROMPT 12 · THE PROOF

The Language Drill

Best for: proving the whole layer without a microphone, then testing it with one.

```
Prove the whole language layer without a microphone, then give me a script to test it with one.

THE DRILL: test_lang_drill.py.
1. Routes, offline: run every sentence from Prompt 10's table, plus traps, through the exact patterns the viewer uses, and print expected against actual. Traps: anything else that starts with "switch to", any phrase another feature might grab first (a voice-changing route can swallow "speak in Spanish"), and "what language is this page in?". Any mismatch fails.
2. The switch, live: POST /lang for all sixteen languages and confirm each returns its own line; POST a made-up language and confirm the refusal.
3. The translator, live: one fixed line into three languages, with first-call and cached milliseconds; confirm English makes no call at all.
4. The tongue, live: ask the brain one English question in three languages, and confirm each answer is in the right language by asking the cheap model which language the text is written in.
5. Put it back: whatever language was active before the drill is active after it.

THE PROBE: ?langprobe=1&mute=1 fakes the language INSIDE that tab only, never on the server, then silently checks: the chip overlaps nothing, the ear stays English, a fixed line arrives translated, a brain answer causes no /xlate call. It writes LANGPROBE n/n into the page title.

PREFLIGHT GROWS: /api/lang answers; an unknown language is refused; /xlate in English mode returns the text untouched with no model call; the translation cache file is readable.

VERIFY: run the drill, open the probe, run preflight, and paste all three summary lines. Then give me my first real-mic script: the exact sentences to say, in order, and what I should see and hear after each one.
```

Make it yours: add one native speaker to the drill. Send them three answers in their language and ask a single question: would you notice this was written by a machine? Fix what they point at, then film.

Seven things I learned making Jarvis multilingual

1 **The brain switched on day one. The furniture did not.**
With the language switched, I asked him to pull up my community notes. He opened them and announced "I've pulled up all the information, sir" in plain English. In a non-English demo, the fixed lines are what give the game away. That is Prompt 09.

2 **The ear and the tongue are two settings.**
I wanted to keep speaking English and hear Spanish back. The browser's recognizer listens for one language at a time, so if it followed the reply language into Spanish, my English would come out garbled. The listening language had to come apart from the reply language. That is Prompt 10.

3 **Translate once, then never again.**
A new line took 1.1 to 1.3 seconds to translate the first time and nothing after that. Pre-translating about thirty fixed lines on every switch meant the "pulled up" line appeared in German in 270 milliseconds.

4 **Never translate the brain twice.**
The brain already writes in the right language. In my test, every translation call came from the warm-up burst and none came from a brain answer. Mark server text as native the moment it lands.

5 **Keep the commands in English.**
In German mode the "pulled up" line became a German sentence with "read it out loud" left in English, on purpose. His ear still listens in English, so those are the words that work.

6 **Phrases collide.**
"Speak in Spanish" was grabbed by my voice-changing route before the language route ever saw it. "Switch to" had to be limited to real language names so "switch to Grok" still swaps the brain. And "how should I respond in Spanish to this client?" has to stay a question.

7 **The lock holds, so film in the right order.**
Locked to German, "translate this into English" gave me the English sentence wrapped in German, and "say that in English, I don't speak German" got a polite refusal in German. Read and translate first, then lock.

Where to go from here

Run Part One in an afternoon, live with the butler for a day, then run Part Two one prompt at a time, in a muted tab, and put the language back after each test. The first time he answers an English question about a Spanish page in German, you will understand why this is a different thing from a translation app.

1 **Post what you built.**
The free community has a thread for every pack. A short clip of your Jarvis answering in your own first language is the best post you can make.

2 **Start with the language you actually speak.**
Your own first language is where you will hear every wrong word. Tune the voice, the confirmation line and the fixed lines there first, then add the rest.

3 **Then give it to the people you serve.**
The same switch, ear and translation layer work in any assistant that talks: a support desk, a booking line, a receptionist. Sixteen languages is sixteen markets.

Want the finished Jarvis without building it?

Inside the paid AI Workshop community you get the full Jarvis build I run every day, with a five-minute install and no code, plus the classroom, the live calls, and a team that helps when something breaks.

Free community: skool.com/aiworkshop-lite
Full workshop: skool.com/aiworkshop
YouTube: Zubair Trabzada | AI Workshop